AMATH 582: HOMEWORK 4

CALEB CRAMER

Applied Mathematics, University of Washington, Seattle, WA
cbcramer@uw.edu

Abstract. We seek to explore supervised machine learning classifiers and compare their performance across a common dataset, namely the wine quality dataset from the UCI ML Repository. We implement a one-hot, affine classifier along with three kernel ridge classifiers and proceed to discuss their performance and accuracy. We find the most accurate classifier to be the hypertuned Laplacian Kernel Ridge Regression, though we are limited by the scarcity of data outside of quality classes 3-8. We also seek to understand the impact each characteristic has on the overall quality of the wine, an interpretability unique to the linear regressor.

1. Introduction and Overview

It may be a bold assumption to think one could find a prescriptive function to describe phenomena of the natural world and every relationship therein. But it is indeed the assumption we make at the outset of each machine learning project, including this one. We hope to find a function with which we can understand and predict the quality of a wine based on 11 characteristics, including various chemical properties, and classify it accordingly.

2. Theoretical Background

2.1. **Machine Learning Framework.** We consider a supervised learning problem where the input space is $\mathcal{X} \subseteq \mathbb{R}^d$ and the output space is a discrete set of classes $\mathcal{Y} = \{1, \ldots, K\}$. We are given a training dataset $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ drawn i.i.d. from an unknown distribution.

To adapt regression techniques for multi-class classification, we employ One-Hot Encoding. We define a mapping $\phi : \mathcal{Y} \to \{0,1\}^K$, where for a sample belonging to class k, the target vector $\mathbf{y}$ has a 1 at the k-th index and 0 elsewhere. This transformation allows us to treat the classification problem as a multi-output regression problem, predicting a vector $\mathbf{y} \in \mathbb{R}^K$ rather than a discrete label. The predicted class is then determined by the index corresponding to the maximum value of the output vector:

$$\hat{y} = \underset{k \in \{1,\ldots,K\}}{\operatorname{argmax}} \; [\hat{\mathbf{t}}]_k \tag{1}$$

2.2. **The Bias-Variance Tradeoff.** The generalization error of a predictive model can be decomposed into three components: bias, variance, and irreducible error. For a squared error loss, the expected error for a given input $\mathbf{x}$ is:

$$\mathbb{E}\left[(y - \hat{f}(\mathbf{x}))^2\right] = \text{Bias}[\hat{f}(\mathbf{x})]^2 + \text{Var}[\hat{f}(\mathbf{x})] + \sigma^2 \tag{2}$$

- **Bias** measures the error introduced by approximating a complex real-world problem with a simplified model. High bias typically leads to underfitting [1].
- **Variance** measures the model's sensitivity to fluctuations in the training set. High variance typically leads to overfitting, where the model captures noise as signal [1].

Date: March 2, 2026.

- **Irreducible Error (σ^2)** is the noise inherent in the data generation process itself [1].

Our modeling strategy aims to minimize the combination of these terms.

2.3. **Linear Regression.** We begin with a linear model, assuming an affine relationship between the inputs and the targets. For a target vector $\mathbf{y} \in \mathbb{R}^K$, the model is defined as:

$$\hat{\mathbf{y}} = \mathbf{\Theta}^T \mathbf{x} + \mathbf{b} \tag{3}$$

where $\mathbf{\Theta} \in \mathbb{R}^{n \times K}$ is the weight matrix and $\mathbf{b} \in \mathbb{R}^K$ is the bias vector. We augment the input vector $\tilde{\mathbf{x}} = [1, \mathbf{x}^T]^T$ and combine the parameters into $\tilde{\mathbf{\Theta}}$, allowing us to write $\hat{\mathbf{y}} = \tilde{\mathbf{\Theta}}^T \tilde{\mathbf{x}}$.

The parameters are estimated by minimizing the Residual Sum of Squares, the optimization problem at the heart of this classifier:

$$\mathcal{L}(\tilde{\mathbf{\Theta}}) = ||\tilde{\mathbf{X}}\tilde{\mathbf{\Theta}} - \mathbf{Y}||_F^2 \tag{4}$$

where $\mathbf{Y}$ is the matrix of target vectors and $\tilde{\mathbf{X}}$ is the design matrix. The closed-form Ordinary Least Squares (OLS) solution is found by setting the gradient to zero:

$$\hat{\mathbf{\Theta}}_{OLS} = (\tilde{\mathbf{X}}^T \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^T \mathbf{Y} \tag{5}$$

2.4. **Kernel Ridge Regression (KRR).** To model non-linear relationships, we employ Kernel Ridge Regression. This method combines Ridge Regression (L_2 regularization) with kernels, allowing the algorithm to operate in a high-dimensional feature space $\mathcal{H}$ without explicitly computing coordinates in that space.

2.4.1. *Formulation.* The objective with L_2 regularization is to find the parameters that minimize:

$$\min_{\mathbf{\Theta}} \sum_{i=1}^{N} ||\mathbf{y}_i - \mathbf{\Theta}^T \Phi(x_i)||^2 + \lambda ||\mathbf{\Theta}||_2^2 \tag{6}$$

Using the Representer Theorem, we express the solution as a linear combination of the training samples in feature space. This leads to the solution for the coefficient matrix $\boldsymbol{\alpha}$:

$$\hat{\boldsymbol{\alpha}} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{Y} \tag{7}$$

We then find the prediction function:

$$f^\star(x) = \sum_{i=1}^{n} \alpha_i^\star \mathbf{K}(x, x_i) = \mathbf{K}(x, \mathbf{X})(\mathbf{K}(\mathbf{X}, \mathbf{X}) + \lambda \mathbf{I})^{-1} \mathbf{Y} \tag{8}$$

2.4.2. *Kernels.* We define three distinct kernel functions $k(\mathbf{x}, \mathbf{x}')$ as defined in scikit-learn's documentation [2]:

1. Radial Basis Function (RBF) Kernel: The RBF kernel measures similarity based on Euclidean distance:

$$k_{RBF}(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma ||\mathbf{x} - \mathbf{x}'||_2^2\right) \tag{9}$$

where γ controls the kernel width.

2. Laplacian Kernel: The Laplacian kernel uses the L_1 distance, which is less sensitive to outliers than the L_2 distance used in RBF:

$$k_{Lap}(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma ||\mathbf{x} - \mathbf{x}'||_1\right) \tag{10}$$

3. Polynomial Kernel: This kernel computes the similarity based on polynomial combinations of features up to degree d:

$$k_{Poly}(\mathbf{x}, \mathbf{x}') = (\gamma \langle \mathbf{x}, \mathbf{x}' \rangle + c_0)^d \tag{11}$$

2.5. **Parameter Tuning and Cross-Validation.** Model performance is contingent on the selection of hyperparameters θ (e.g., λ, γ, d). We employ k-fold Cross-Validation to estimate the generalization error. The training data is split into k folds. For each fold j, the model is trained on the remaining $k-1$ folds and evaluated on fold j. The cross-validation score is the average error across all folds:

$$CV(\theta) = \frac{1}{k}\sum_{j=1}^{k} E_j(\theta) \tag{12}$$

We select the hyperparameters that minimize this score: $\hat{\theta} = \text{argmin}_\theta CV(\theta)$.

3. Algorithm Implementation and Development

3.1. **Read and Split Data.** The data is read from a CSV file containing 1594 wines and their 11 characteristics: (fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol). It is imperative to split the dataset into a training and testing set. Keeping these separate ensures we have independent data to check our function against after training. We chose to split the data into 1115 samples for training and 479 for testing.

3.2. **Scale Characteristics.** Since we are applying numerical functions to this data, it is important to normalize the inputs so that one characteristic of the wine does not dominate the algorithm. We achieved this by calling `sklearn.preprocessing.StandardScaler()`, which scales the numerical range of each column to have a mean of 0 and a standard deviation of 1 ($z = \frac{x-\mu}{\sigma}$).

3.3. **Encode Classes.** Since we intend to classify the wines into quality levels indicated by integers from 0-10, we utilized one-hot encoding. We relied on sklearn's `OneHotEncoder` for this functionality. It creates a new column for each of the classes in the dataset and records a 1 in the corresponding column. For example, a wine with a quality of 8 would have a 1 in the newly created `class_8` column.

3.4. **Tune Parameters.** As shown in Section 2.5, we must tune the models to optimize our parameters, λ, σ, c, and d. The `sklearn` library accepts these parameters as α and γ. We created a grid of these parameters to search, cross-validating 10 folds (2.5) with each combination. For each iteration, we computed the negative mean squared error and searched the grid for the maximum value. This yielded the values used in our final fitting and prediction. To see contour plots for the three kernel methods, review Appendix A Figure 1.

3.5. **Fit and Predict.** With the optimal parameters, we fit the model to the training set and predicted the classes of the test wines. Due to the behavior of the sklearn encoder, it optimally creates columns only for the classes detected in the training set (in our case, 3-8). Consequently, we added an offset to the result of the predictions to convert the index returned by the argmax function into the correct class value.

3.6. **Compare Accuracy.** Finally, we compared the results achieved by each algorithm to the true labels of the test dataset. The average number of correct predictions represents the accuracy of our algorithm. See Section 4 for a further discussion of results.

4. Computational Results

The final accuracy for each implementation is as follows:

Algorithm	λ	σ	c	d	Accuracy
Affine	N/A	N/A	N/A	N/A	52.61%
Gaussian Kernel	5.62	3.0	N/A	N/A	58.66%
Laplacian Kernel	0.1778	3.0	N/A	N/A	64.72%
Polynomial Kernel (MSE fit)	100	N/A	5.0	3.0	56.78%
Polynomial Kernel (Custom fit)	1.122	N/A	1.0	3.0	58.66%

Table 1. Accuracy Results

Some noteworthy observations from Table 1 include:

- The Laplacian Kernel performed best of all the options. This is likely due to its L_1-norm implementation being more resistant to outliers than the RBF kernel. This makes intuitive sense for a metric based on human decision-making, where a small transition in acidity may be acceptable up to a point, after which it becomes unbearable and sharply affects the wine's rating.
- The difference of 2% in the Polynomial Kernel implementations was due to a custom scoring method suggested by a colleague, which we implemented during cross-validation. Instead of scoring the results (discrete classes) with the typical Mean Squared Error, it is more logical to score them with a discrete valuation. We built a custom scoring function to count the number of correct answers and take the average during each fold. This led to optimized parameters, which in turn led to the accuracy increase.
- The Affine implementation performed the worst of all the algorithms. This is to be expected for the affine model since there were no parameters to tune and the nature of the data was resistant to this linear classifier. However, it serves as an excellent baseline and allows us some insight into the impact of each characteristic on each class, as seen in Table 2. As an example, one way to understand this table is:

$$\text{Score}_{5.0} = 0.413 - 0.071(x_{\text{fixed acidity}}) + 0.067(x_{\text{volatile acidity}}) + \ldots - 0.122(x_{\text{alcohol}}) \tag{13}$$

- Helpful visualizations of the Confusion Matrix can be found in Appendix A Figures 2a, 2b, 2c, and 2d.

Variable	quality_3.0	quality_4.0	quality_5.0	quality_6.0	quality_7.0	quality_8.0
constant	0.007	0.034	0.413	0.413	0.126	0.007
fixed acidity	0.016	0.032	-0.071	-0.022	0.050	-0.005
volatile acidity	0.020	0.040	0.067	-0.087	-0.043	0.004
citric acid	0.009	0.015	0.043	-0.089	0.015	0.007
residual sugar	0.002	0.025	-0.024	-0.035	0.032	-0.000
chlorides	0.007	0.006	0.038	-0.014	-0.031	-0.006
free sulfur dioxide	0.004	-0.008	-0.060	0.060	0.005	-0.001
total sulfur dioxide	-0.007	-0.010	0.158	-0.105	-0.035	-0.002
density	-0.011	-0.046	0.046	0.067	-0.053	-0.002
pH	0.013	0.030	-0.038	-0.006	0.008	-0.007
sulphates	-0.002	0.005	-0.112	0.050	0.054	0.004
alcohol	-0.008	-0.027	-0.122	0.084	0.067	0.006

Table 2. Correlation coefficients from affine regression indicating the importance of each characteristic for each class

Finally, we estimate the qualities of five wines with no true ratings yet, based off our optimized models. The three models are in consensus for the first three wines but differ minorly on the final two. The two lowest performers agree with each other but disagree with the highest performer in hindsight. Which should we believe?

ID	Fixed Acidity	Volatile Acidity	Citric Acid	Resid. Sugar	Chlor.	Free SO_2	Total SO_2	Dens.	pH	Sulph.	Alc.	Pred. (RBF)	Pred. (Lap.)	Pred. (Poly)
0	6.2	0.21	0.29	1.6	0.039	24	92	0.99114	3.27	0.50	11.2	6	6	6
1	6.6	0.32	0.36	8.0	0.047	57	168	0.99490	3.15	0.46	9.6	5	5	5
2	6.5	0.24	0.19	1.2	0.041	30	111	0.99254	2.99	0.46	9.4	5	5	5
3	5.5	0.29	0.30	1.1	0.022	20	110	0.98869	3.34	0.38	12.8	6	7	6
4	6.0	0.21	0.38	0.8	0.020	22	98	0.98941	3.26	0.32	11.8	6	7	6

Table 3. Prediction of New Wine Quality for RBF, Laplacian, and Polynomial Kernels based on characteristics

5. Summary and Conclusions

In this study, we explored the application of supervised machine learning techniques to classify wine quality based on physicochemical properties. We implemented and compared an Affine Linear Regressor against Kernel Ridge Regression models utilizing RBF, Laplacian, and Polynomial kernels.

Our results demonstrate that non-linear models significantly outperform the linear baseline, validating the hypothesis that the relationship between chemical features and wine quality is complex and non-affine. Specifically, the Laplacian Kernel classifier achieved the highest accuracy of 64.72%. This superior performance is likely attributed to the Laplacian kernel's use of the L_1 metric, which provides greater robustness to outliers and better models the sharp "threshold" effects often observed in quality grading, where a specific defect can abruptly degrade a wine's score.

While the Affine model yielded the lowest accuracy (52.61%), it provided valuable interpretability through feature weights, allowing us to quantify the impact of specific characteristics—such as alcohol content and volatile acidity—on specific quality tiers.

A significant limitation encountered was the class imbalance within the UCI dataset, which is heavily concentrated around average quality ratings (classes 5 and 6) with sparse data for extreme qualities (3, 4, 7, 8). This constrained our ability to effectively train and evaluate the models on the full spectrum of wine qualities. Future work would benefit from techniques to address this imbalance, such as class weighting, or the exploration of ensemble methods to further improve classification accuracy.

Another area for future improvement is the efficiency of implementation which held us back from testing hypotheses, like the scoring, rapidly. Using `sklearn`'s functions and optimal implementations would free us to experiment further with those sorts of theories.

Acknowledgements

I am thankful to Artur, Ali, and Kyle for enlightening discussion especially on the tuning of parameters. Assistance with LaTeX formatting was provided by Google and Gemini. I also heavily relied on Nathan Kutz's book and the `sklearn` documentation.

References

[1] J. N. Kutz. Data-driven modeling and scientific computation methods for integrating dynamics of complex systems and big data.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Appendix A. Figures

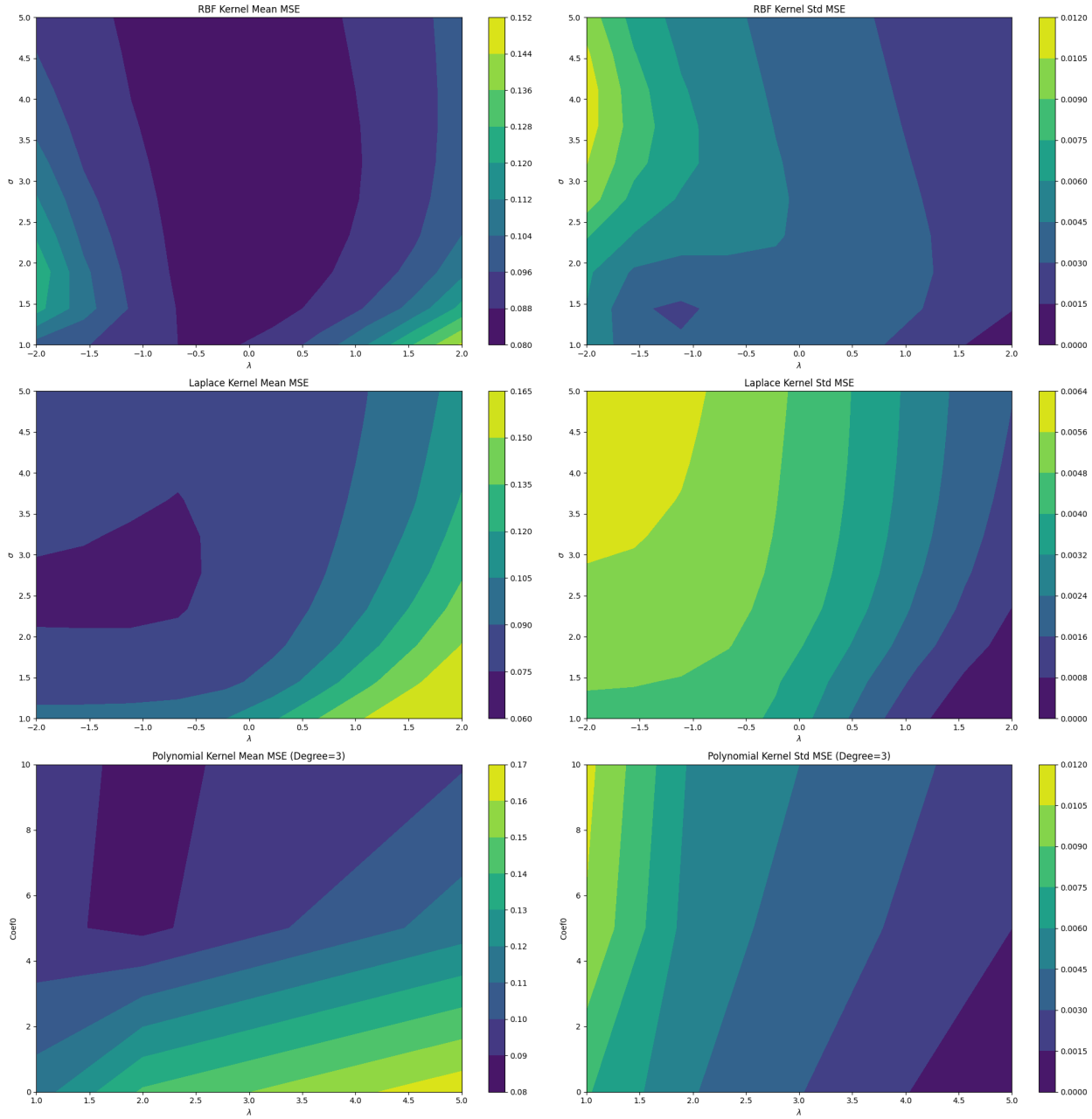
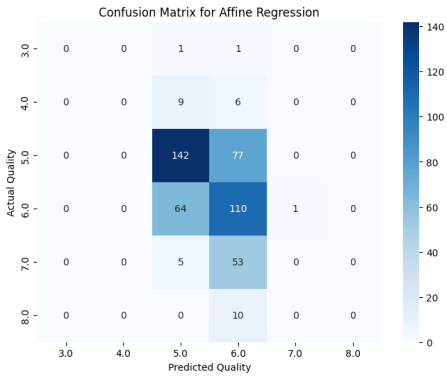
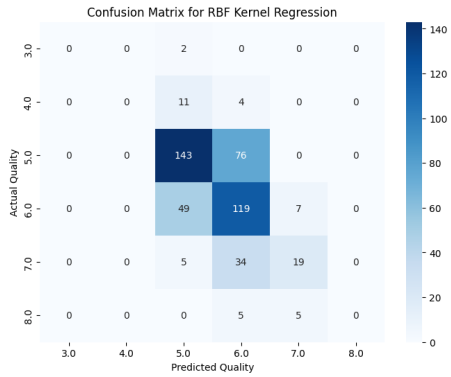


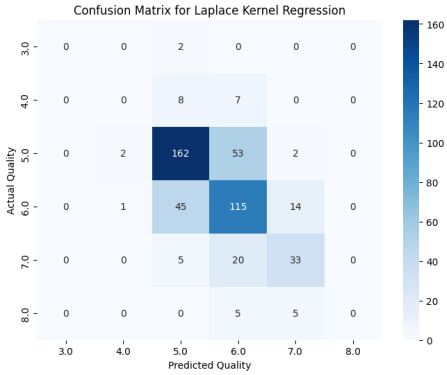
Figure 1. Contour plots help to visualize the parameter search and tuning. In each of the above cases, we choose the largest regularizer that also maintains the lowest error. This encourages a simple model while minimizing the error.



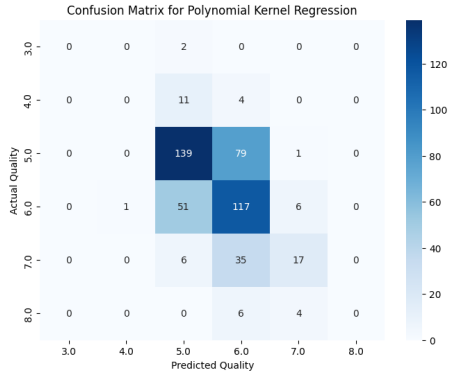
(A) Accuracy = 52.6%.



(B) Accuracy = 58.6%.



(C) Accuracy = 64.7%.



(D) Accuracy = 56.7%.

FIGURE 2. Confusion Matrices for each of the four implemented algorithms described previously.